

Summary: Analysis of Flow Control in C — Nested Loops, While Structures, and Jump Statements

This summary synthesizes the technical insights from Lecture 17 on C Programming, focusing on dependent nested loops, iteration structures (for, while, do-while), and jump statements (break and continue). The core emphasis is on understanding how loop execution depends on mathematical logic, selecting the appropriate loop structure based on iteration predictability, and controlling execution flow using jump statements.

1. Analysis of Dependent Nested Loops

Dependent nested loops are defined by an inner loop whose execution range or termination condition depends on the current value of the outer loop variable. Unlike independent loops, the total execution count varies across iterations and must be computed mathematically.

1.1 Case Study: Linear Dependency

- Outer loop variable: i ranges from 1 to n
- Inner loop variable: j ranges from 1 to i

Execution Pattern

$1 + 2 + 3 + \dots + n$

Total Executions

$n(n + 1) / 2$

This represents the sum of the first n natural numbers.

1.2 Advanced Range Dependencies

The source includes examples where the inner loop range expands proportionally with the outer loop variable.

Outer Loop (i)	Inner Loop (j) Range	Execution Count per i	Total Execution Formula
----------------	----------------------	-----------------------	-------------------------

1 to n	i to $2 \times i$	$(2i - i + 1) = i + 1$	$n(n + 3) / 2$
1 to n	i to $3 \times i$	$(3i - i + 1) = 2i + 1$	$n(n + 2)$

General Rule

For any range $[a, b]$, the number of iterations is:

$$b - a + 1$$

2. Mathematical Foundations for Loop Analysis

Accurate loop analysis relies on summation formulas and sequence patterns.

2.1 Summation Formulas

- Sum of first k integers

$$k(k + 1) / 2$$

- Sum of squares up to k^2

$$k(k + 1)(2k + 1) / 6$$

- Sum of cubes up to k^3

$$[k(k + 1) / 2]^2$$

2.2 Progressions

Arithmetic Progression (AP)

A sequence with a constant difference between terms.

- Sum of n terms

$$S_n = n / 2 \times (\text{First Term} + \text{Last Term})$$

Geometric Progression (GP)

A sequence with a constant common ratio r .

- Sum of n terms

$$S_n = a(r^n - 1) / (r - 1)$$

- **Infinite GP ($|r| < 1$)**

$$S^{\infty} = a / (1 - r)$$

3. Comparison of Iterative Structures

C provides three primary loop structures, each suited to different logical requirements.

3.1 For Loop vs. While Loop

- **For Loop**
 - Preferred when the number of iterations is known in advance.
 - Example use case: Printing a multiplication table with a fixed number of entries.
 - All three expressions (initialization, condition, increment) are optional.
- **While Loop**
 - Preferred when the number of iterations depends on a condition.
 - Example use case: Processing digits of a number until it becomes zero.
 - The condition expression is mandatory and cannot be omitted.

3.2 Entry-Controlled vs. Exit-Controlled Loops

- **Entry-Controlled Loops (For, While)**
 - Condition is checked before execution.
 - If the condition is false initially, the loop executes zero times.
- **Exit-Controlled Loop (Do-While)**
 - Code executes first, condition is checked afterward.
 - Guarantees at least one execution.
 - Commonly used in menu-driven programs where the menu must be displayed at least once.

4. Practical Implementation: Digit Counting Using While Loop

A common application of the while loop is counting the number of digits in an integer n .

Algorithm Logic

1. Initialize counter: $C = 0$
2. While $n \neq 0$:

Perform integer division:

$$n = n / 10$$

○

Increment counter:

C++

○

3. Final value of `C` gives the number of digits.

Examples

- `n = 128`
Division occurs 3 times → Result = 3
- `n = 7264`
Division occurs 4 times → Result = 4

5. Jump Statements: Break and Continue

Jump statements provide fine-grained control over loop execution.

5.1 Continue Statement

- **Function**
Skips the remaining statements in the current iteration and proceeds to the next iteration.
- In a for loop, control jumps directly to the increment step.

Example Behavior

Condition:

```
if (i % 3 == 0) continue;
```

-
- Loop: 1 to 10

Output:

1, 2, 4, 5, 7, 8, 10

-

5.2 Break Statement

- **Function**
Immediately terminates the entire loop and transfers control to the statement following the loop.

Analogy

- Continue: Skipping a difficult question and moving to the next.

- Break: Leaving the exam hall entirely.

Example Behavior

Condition:

```
if (i % 3 == 0) break;
```

-
- Loop: 1 to 10

Output:

1, 2

-

6. Conclusion

Effective flow control in C depends on understanding both loop mechanics and the mathematical logic behind execution counts. Dependent nested loops require summation analysis, while loop selection depends on iteration predictability and execution guarantees. The strategic use of **break** and **continue** further enhances control by allowing selective iteration skipping or immediate loop termination. Together, these constructs form the foundation of structured and efficient C programming.